Code Evidence Screenshots

1. Frontend (What You See)

src/app/(dashboard)/dashboard/page.tsx

```
// Next.js App Router — Dashboard Page Component
'use client';

import { useEffect, useState } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { Loader2, AlertCircle } from 'lucide-react';
import { useSession } from 'next-auth/react';
import type { DashboardStats } from '@/types/statistics';

export default function SchoolWideDashboard() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const { data: session } = useSession();
  const [stats, setStats] = useState<DashboardStats | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchDashboardStats();
  }, []);

  const fetchDashboardStats = async () => {
    try {
      const response = await fetch('/api/dashboard/stats');
      if (response.ok) {
        const data = await response.json();
        setStats(data);
      }
    } catch (error) {
      console.error('Error fetching dashboard stats:', error);
    } finally {
      setLoading(false);
    }
  };
```

Image 1.1 — Next.js App Router: Dashboard Page with Data Fetching

src/hooks/useAlerts.ts

```
import useSWR from 'swr';
import type { Alert } from '@/types/alert';

const fetcher = async (url: string) => {
  const res = await fetch(url);
  if (!res.ok) throw new Error('Failed to fetch alerts');
  return res.json();
};

interface UseAlertsOptions {
  unreadOnly?: boolean;
  refreshInterval?: number;
}

export function useAlerts(options: UseAlertsOptions = {}) {
  const {
    unreadOnly = false,
    refreshInterval = 5000 // Poll every 5 seconds for real-time notifications
  } = options;

  const url = `/api/alerts${unreadOnly ? '?unread=true' : ''}`;

  const { data, error, mutate, isLoading } = useSWR<Alert[]>(
    url,
    fetcher,
    { refreshInterval }  // Auto re-fetches every 5 seconds
  );

  const markAsRead = async (alertId: string) => {
    const res = await fetch(`/api/alerts/${alertId}/read`, {
      method: 'PUT',
    });
    if (!res.ok) throw new Error('Failed to mark alert as read');
    mutate();  // Triggers SWR re-validation (optimistic update)
  };

  const resolveAlert = async (alertId: string, notes?: string) => {
    const res = await fetch(`/api/alerts/${alertId}/resolve`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
    });
    if (!res.ok) throw new Error('Failed to resolve alert');
    mutate();
  };
```

Image 1.2 — React: Custom Hook with Real-Time SWR Polling & Optimistic Updates

src/components/layout/sidebar.tsx

```
// Sidebar Navigation with Tailwind CSS Medical Theme
function NavContent({ navItems, pathname, onMobileClose, onLogout }: NavContentProps) {
  return (
    <>
      {/* Logo Section */}
      <div className="p-6 border-b border-gray-100">
        <Link href="/dashboard" className="flex items-center gap-3">
          <Image src="/logo.png" alt="MED-Alert" width={40} height={40} />
          <div>
            <h1 className="text-xl font-bold" style={{ color: '#C41E3A' }}>
              MED-Alert
            </h1>
            <p className="text-xs text-gray-500">School Clinic System</p>
          </div>
        </Link>
      </div>

      {/* Navigation Items */}
      <nav className="flex-1 min-h-0 p-4 space-y-2 overflow-y-auto">
        {navItems.map((item) => {
          const isActive = pathname === item.href;
          return (
            <Link
              key={item.href}
              href={item.href}
              className={cn(
                'flex items-center gap-3 px-4 py-3 rounded-xl font-medium transition-al
                isActive
                  ? 'text-white shadow-lg'
                  : 'text-gray-600 hover:bg-red-50 hover:text-[#C41E3A]'
              )}
              style={isActive ? {
                background: 'linear-gradient(135deg, #C41E3A 0%, #E63946 100%)',
                boxShadow: '0 4px 12px rgba(196, 30, 58, 0.3)',
              } : undefined}
            >
              <Icon className="w-5 h-5" />
              <span>{item.label}</span>
            </Link>
          );
        })}
      </nav>

      {/* Logout Button */}
      <div className="p-4 border-t border-gray-100">
        <button
          onClick={onLogout}
```

```
            className="flex items-center gap-3 px-4 py-3 rounded-xl font-medium text-red-
          >
            <LogOut className="w-5 h-5" />
            <span>Logout</span>
          </button>
        </div>
      </>
    );
  }
```

Image 1.3 — Tailwind CSS: Medical Red Theme, Responsive Layout & Gradient Styling

2. Backend (The Brain & Storage)

src/lib/db.ts

```
import mysql, { Pool, PoolConnection, RowDataPacket, ResultSetHeader } from 'mysql2/pro

// Create connection pool with Aiven-compatible settings
const pool: Pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT || '3306'),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 5,  // Aiven free tier limit
  queueLimit: 0,
  enableKeepAlive: true,
  ssl: {
    rejectUnauthorized: false
  }
});

/**
 * Execute a query that returns multiple rows
 * @param sql - SQL query string with ? placeholders
 * @param params - Array of parameter values
 * @returns Array of results
 */
export async function query<T = RowDataPacket>(
  sql: string,
  params?: (string | number | boolean | Date | null)[]
): Promise<T[]> {
  const [results] = await pool.execute<RowDataPacket[]>(sql, params);
  return results as T[];
}

/**
 * Execute multiple queries in a transaction
 * Automatically commits on success, rolls back on error
 */
export async function transaction<T>(
  callback: (connection: PoolConnection) => Promise<T>
): Promise<T> {
  const connection = await pool.getConnection();
  await connection.beginTransaction();

  try {
    const result = await callback(connection);
    await connection.commit();
    return result;
  } catch (error) {
```

```
    await connection.rollback();
    throw error;
  } finally {
    connection.release();
  }
}
```

Image 2.1 — MySQL: Database Connection Pool, Query Helpers & Transaction Wrapper

src/lib/queries/students.ts

```
import { query, queryOne, transaction } from '../db';
import type { Student, StudentSearchOptions, StudentListResult } from '@/types/student'

/**
 * Find students with search, pagination, and filters
 */
export async function findStudents(options: StudentSearchOptions): Promise<StudentListR
  const { page = 1, limit = 20, search = '', grade = '' } = options;
  const offset = (page - 1) * limit;

  const whereClauses: string[] = [];
  const params: (string | number)[] = [];

  // Search filter
  if (search) {
    whereClauses.push(`(u.first_name LIKE ? OR u.last_name LIKE ? OR s.lrn LIKE ?)`);
    const searchPattern = `%${search}%`;
    params.push(searchPattern, searchPattern, searchPattern);
  }

  // Grade filter
  if (grade) {
    whereClauses.push('s.grade_level = ?');
    params.push(grade);
  }

  const whereClause = whereClauses.length > 0
    ? `WHERE ${whereClauses.join(' AND ')}` : '';

  // SQL Query with JOIN, dynamic WHERE, and pagination
  const sql = `
    SELECT
      s.id, s.user_id as userId,
      u.first_name as firstName, u.last_name as lastName,
      s.age, s.date_of_birth as dateOfBirth, s.sex,
      s.grade_level as gradeLevel, s.section,
      s.lrn, s.student_number as studentNumber,
      s.parent_guardian_name as parentGuardianName,
      s.bmi, s.health_history as healthHistory
    FROM students s
    JOIN users u ON s.user_id = u.id
    ${whereClause}
    ORDER BY u.last_name, u.first_name
    LIMIT ${limit} OFFSET ${offset}
  `;

  const students = await query<Student>(sql, params);
```

```
  // Get total count for pagination
  const countResult = await queryOne<{ total: number }>(
    `SELECT COUNT(*) as total FROM students s JOIN users u ON s.user_id = u.id ${whereC
    params
  );

  return { students, total: countResult?.total || 0, page, limit };
}
```

Image 2.2 — MySQL: SQL Query with JOIN, Dynamic Filtering & Pagination

src/types/student.ts & src/types/user.ts

```ts
// ─── src/types/student.ts ───

export type GradeLevel = '7' | '8' | '9' | '10' | '11' | '12' | 'Non-Graded';
export type Sex = 'MALE' | 'FEMALE';

export interface Student {
  id: string;
  userId: string;

  // Student Identification
  lrn: string;            // Learner Reference Number
  studentNumber: string;

  // Personal Information
  firstName: string;
  lastName: string;
  middleName?: string;     // optional
  dateOfBirth: string;     // ISO date string
  age: number;
  sex: Sex;

  // Academic Information
  gradeLevel: GradeLevel;
  section: string;

  // Medical Information
  bmi?: number;
  healthHistory?: string;

  // System Fields
  createdAt: string;
  updatedAt: string;
}

// ─── src/types/user.ts ───

import { RowDataPacket } from 'mysql2';

export type UserRole = 'SUPER_ADMIN' | 'ADMIN' | 'PATIENT';

export interface User extends RowDataPacket {
  id: string;
  username: string;
  email: string | null;
  password: string;
  role: UserRole;
  first_name: string;
```

```
  last_name: string;
  is_active: boolean;
  last_login: Date | null;
  created_at: Date;
  updated_at: Date;
}

export interface SessionUser {
  id: string;
  name?: string | null;
  email?: string | null;
  role: UserRole;
}
```

Image 2.3 — TypeScript: Interfaces, Union Types & Type-Safe Definitions

3. Security (The Guard)

src/lib/auth.ts

```
import NextAuth from 'next-auth';
import Credentials from 'next-auth/providers/credentials';
import bcrypt from 'bcryptjs';
import { findUserByUsername, updateUserLastLogin } from './queries/users';
import type { UserRole } from '@/types/user';

export const { handlers, signIn, signOut, auth } = NextAuth({
  providers: [
    Credentials({
      credentials: {
        username: { label: 'Username', type: 'text' },
        password: { label: 'Password', type: 'password' },
      },
      async authorize(credentials) {
        if (!credentials?.username || !credentials?.password) return null;

        const username = credentials.username as string;
        const password = credentials.password as string;

        // Find user by username
        const user = await findUserByUsername(username);
        if (!user) return null;

        // Check if account is disabled
        if (!user.is_active) throw new Error('ACCOUNT_DISABLED');

        // Verify password with bcrypt
        const isValidPassword = await bcrypt.compare(password, user.password);
        if (!isValidPassword) return null;

        // Update last login timestamp
        await updateUserLastLogin(user.id);

        return {
          id: user.id,
          name: `${user.first_name} ${user.last_name}`,
          email: user.email,
          role: user.role,
        };
      },
    }),
  ],
  callbacks: {
    async jwt({ token, user }) {
      // Add custom fields to JWT token on first sign in
      if (user) {
        token.id = user.id as string;
```

```
        token.role = user.role;
      }
      return token;
    },
    async session({ session, token }) {
      // Add custom fields to session from JWT
      if (session.user) {
        session.user.id = token.id as string;
        session.user.role = token.role as UserRole;
      }
      return session;
    },
  },
  pages: { signIn: '/login' },
  session: {
    strategy: 'jwt',
    maxAge: 24 * 60 * 60,  // 24 hours
  },
  trustHost: true,
});

/**
 * Role hierarchy for permission checks
 */
export const roleHierarchy = {
  SUPER_ADMIN: ['SUPER_ADMIN', 'ADMIN', 'PATIENT'],
  ADMIN: ['ADMIN', 'PATIENT'],
  PATIENT: ['PATIENT'],
};
```

Image 3.1 — NextAuth.js: Authentication with Credentials Provider & JWT Sessions

src/app/api/users/route.ts

```
// Bcrypt Password Hashing — User Creation API

import bcrypt from 'bcryptjs';
import { createUser } from '@/lib/queries/users';
import { logAction } from '@/lib/audit-logger';

export async function POST(request: NextRequest) {
  // ... auth & validation checks ...

  // Hash password with bcrypt (cost factor 10)
  const hashedPassword = await bcrypt.hash(validatedData.password, 10);

  // Create user with hashed password
  const userId = await createUser({
    username: validatedData.username,
    email: validatedData.email,
    password: hashedPassword,   // stored as bcrypt hash, never plaintext
    role: validatedData.role,
    firstName: validatedData.firstName,
    lastName: validatedData.lastName,
  });

  // Log user creation for compliance audit trail
  await logAction(session.user.id, 'CREATE', 'users', userId, null, {
    username: validatedData.username,
    role: validatedData.role,
  });

  return NextResponse.json({ userId }, { status: 201 });
}

// ─── Password Verification (from src/lib/auth.ts) ───

// Bcrypt compares plaintext password against stored hash
const isValidPassword = await bcrypt.compare(password, user.password);
if (!isValidPassword) return null;  // Generic error for security
```

Image 3.2 — Bcrypt: Password Hashing (Registration) & Verification (Login)

src/lib/rbac.ts

```ts
/**
 * Role-Based Access Control (RBAC) Helpers
 * Authorization utilities for managing user permissions and access control
 */

export type UserRole = 'SUPER_ADMIN' | 'ADMIN' | 'PATIENT';

/**
 * Check if user has one of the required roles
 */
export function hasRole(userRole: UserRole, allowedRoles: UserRole[]): boolean {
  return allowedRoles.includes(userRole);
}

/**
 * Check if user is an administrator (SUPER_ADMIN or ADMIN)
 */
export function isAdmin(role: UserRole): boolean {
  return role === 'SUPER_ADMIN' || role === 'ADMIN';
}

/**
 * Check if user is a patient
 */
export function isPatient(role: UserRole): boolean {
  return role === 'PATIENT';
}

/**
 * Check if user can access a specific student record
 * - SUPER_ADMIN and ADMIN can access any student
 * - PATIENT can only access their own student record
 */
export function canAccessStudent(
  userRole: UserRole,
  userId: string,
  studentUserId?: string | null
): boolean {
  // Admins can access all students
  if (isAdmin(userRole)) return true;

  // Patients can only access their own record
  if (isPatient(userRole)) return userId === studentUserId;

  // Default: deny access
```

```
  return false;
}
```

Image 3.3 — RBAC: Role-Based Access Control with Ownership Checks

src/middleware.ts

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { auth } from '@/lib/auth';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Get the session (Edge Runtime compatible)
  const session = await auth();
  const isLoggedIn = !!session?.user;
  const userRole = session?.user?.role;

  // Redirect to login if not authenticated
  if (!isLoggedIn) {
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('callbackUrl', pathname);
    return NextResponse.redirect(loginUrl);
  }

  // Admin-only routes (block patients)
  const adminOnlyRoutes = ['/patients', '/registration', '/statistics', '/alerts', '/me
  const isAdminRoute = adminOnlyRoutes.some(route => pathname.startsWith(route));

  if (isAdminRoute && userRole === 'PATIENT') {
    return NextResponse.redirect(new URL('/patient-portal', request.url));
  }

  // Dashboard route - redirect patients to patient portal
  if (pathname.startsWith('/dashboard') && userRole === 'PATIENT') {
    return NextResponse.redirect(new URL('/patient-portal', request.url));
  }

  return NextResponse.next();
}
```

Image 3.4 — Route Protection Middleware: Authentication & Role-Based Access

4. Infrastructure & Code Management

src/lib/db.ts — Aiven Cloud MySQL Configuration

```
import mysql, { Pool } from 'mysql2/promise';

// Create connection pool with Aiven Cloud MySQL settings
const pool: Pool = mysql.createPool({
  host: process.env.DB_HOST,          // *.aivencloud.com
  port: parseInt(process.env.DB_PORT || '3306'),
  user: process.env.DB_USER,          // avnadmin
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,      // defaultdb
  waitForConnections: true,
  connectionLimit: 5,                // Aiven free tier connection limit
  enableKeepAlive: true,
  ssl: {
    rejectUnauthorized: false        // Required for Aiven's managed SSL
  }
});

// Verify database connection on startup
pool.getConnection()
  .then(connection => {
    console.log('Database connected successfully');
    connection.release();
  })
  .catch(err => {
    console.error('Database connection failed:', err.message);
  });
```

Image 4.1 — Aiven: Cloud MySQL Database Connection with SSL

next.config.ts — Vercel Deployment Configuration

```typescript
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  // Enable compression for faster loading
  compress: true,

  // Image optimization (Vercel Image CDN)
  images: {
    formats: ['image/avif', 'image/webp'],
    deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048],
    minimumCacheTTL: 60,
  },

  // Security headers (applied at Vercel edge/CDN level)
  async headers() {
    return [{
      source: '/:path*',
      headers: [
        { key: 'X-DNS-Prefetch-Control', value: 'on' },
        { key: 'X-Frame-Options',        value: 'SAMEORIGIN' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'Referrer-Policy',       value: 'strict-origin-when-cross-origin' },
      ],
    }];
  },

  reactStrictMode: true,
  poweredByHeader: false,  // Remove X-Powered-By for security

  experimental: {
    optimizeCss: true,
    optimizePackageImports: ['lucide-react', 'recharts'],
  },
};

export default nextConfig;
```

Image 4.2 — Vercel: Next.js Deployment Config with Security Headers & Optimization

package.json — Project Dependencies (Technology Stack)

```
{
  "name": "medicare-app",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "bcryptjs": "^3.0.3",            // Password encryption
    "lucide-react": "^0.562.0",       // Icons
    "mysql2": "^3.16.0",              // MySQL database driver
    "next": "16.1.1",                // Next.js framework
    "next-auth": "^5.0.0-beta.30",   // Authentication (NextAuth v5)
    "react": "19.2.3",               // React UI library
    "react-dom": "19.2.3",           // React DOM rendering
    "react-hook-form": "^7.70.0",    // Form management
    "recharts": "^3.6.0",            // Charts & statistics
    "swr": "^2.3.8",                 // Real-time data fetching
    "zod": "^4.3.5",                 // Schema validation
    "@radix-ui/react-dialog": "^1.1.15",  // UI components
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "tailwind-merge": "^3.4.0",      // Tailwind CSS utilities
    "jspdf": "^4.2.0",               // PDF generation
    "date-fns": "^4.1.0"             // Date formatting
  },
  "devDependencies": {
    "tailwindcss": "^4",             // Tailwind CSS framework
    "typescript": "^5",              // TypeScript compiler
    "@types/react": "^19",          // TypeScript type definitions
    "eslint": "^9",                 // Code linting
    "eslint-config-next": "16.1.1"  // Next.js ESLint rules
  }
}
```

Image 4.3 — GitHub Repository: Project Dependencies & Technology Stack